Team Assignment Members:

Aidralieva Salma
Umarov Azamzhan
Attokurov Zarylbek

Question 1.

Task: Run the code as-is. Before any pages are loaded, answer:

1. How many physical frames exist in total?
- 8
2. How many are already occupied, and by what?
- Occupied: 3 frames. "OS" is in frames 1 and 6, and "AppX" is in frame 3.
3. How many frames are available for the program?
- 5, which are 0, 2, 4, 5 and 7.

Then modify: Change “physical_memory” so that only “one” frame is free, and predict what happens when the program tries to load 4 pages.

- Page A takes the single free frame. After that available_frames is empty.
  - The if available_frames: check is false for B, C and D, so they are silently skipped.
  - page_table ends up with only one entry, for example {0: 5}.
  - The program then runs page_table[2] and crashes with a KeyError, because page 2 was never loaded.
  - In a real OS this is "out of memory". The OS would have to swap pages to disk.

Question 2

Task: Suppose after running the code, the page table is:

- page_table = {0: 4, 1: 0, 2: 7, 3: 2}

and physical memory is:

- physical_memory = ['B', 'OS', 'D', 'AppX', 'A', None, 'OS', None]

1. What does the program get when it asks for “logical_address = 3”?
- D
2. What about “logical_address = 1”?
- B

3. Modify: Change “logical_address = 5”. What happens, and why?
- KeyError: 5, we don’t have that key, its an invalid address

Question 3

Task: The code uses random.choice(available_frames).

1. Run the code three times. Does the page table look the same each time? Why or why not?
- No, because of random
2. Modify: Add “random.seed(42)” at the top of the code. Run it three times. What changes?
- It returns the same numbers and sequence of them each time
3. Why is reproducibility important when testing memory allocation algorithms?
- It helps you find and fix errors by keeping your test data the same every time you run a broken script.

Question 4

Task: Consider two allocation strategies for the same initial memory [None, 'OS', None, 'AppX', None, None, 'OS', None]:

- Strategy A (current code): Randomly choose any available frame.
- Strategy B: Always choose the lowest-numbered available frame (first-fit).

1. Modify the code to implement Strategy B (physical_frame_num = available_frames[0]).
2. Compare the resulting page tables for both strategies.
3. Which strategy produces a more compact physical memory layout? Does compactness matter here?
- Strategy B. Its used frames are packed toward the start, and the free space is one block at the end (frame 7). Strategy A leaves a hole at frame 4 in the middle.
  No compactness doesnt matter much. Paging lets any page go in any frame, so scattered pages work just as well. It would matter for contiguous allocation.
4. Does either strategy cause external fragmentation? Explain.
- External fragmentation: neither strategy causes it. External fragmentation means there is enough total free memory but no single contiguous block big enough. Paging avoids this because every frame is the same size and pages dont need to be contiguous. Any free frame will do.

Question 5

Task: Run the code exactly as written, then answer the following.

Look at this line:

physical_frame_num = random.choice(available_frames)

Now consider the for loop. Is it possible for two different logical pages (e.g., page 1 and page 3) to end up in the same physical frame? Explain why or why not by tracing what happens to available_frames.

- No because we remove the available_frame after allocating it using .remove(physical_frame_num)

Comment out this line:

available_frames.remove(physical_frame_num)

Run the code again. What happens to page_table and physical_memory? Why?

- Now multiple logical pages can end up in the same frame, so we will rewrite the memory

Restore the “remove” line (based on the point 2), but change “random.choice(available_frames)” to “available_frames[-1]”. Run the code three times. Is the output the same every time? What does this tell you about the role of randomness in the simulation?

- Yes, the output is identical across every run. Because available_frames[-1] deterministically picks the last element each time, pages A, B, C, and D will always map to the same frames.
  Randomness simulates the unpredictable nature of real OS memory allocation, showing that paging works regardless of which physical frames are granted. Removing randomness makes the allocation fully deterministic and reproducible.

Suppose you run the original code and get:

page_table = {0: 4, 1: 0, 2: 7, 3: 2}

The program now asks for “logical_address = 1”. It finds “B”. Now suppose the program asks for “logical_address = 4”. What happens, and what does this tell you about the difference between a “logical page number” and a “physical frame number”?

- A request for logical page 4 is invalid here because the program only has logical pages 0–3. While Frame 4 exists in physical memory (holding page 'A'), Logical Page 4 does not exist in the program's address space.